

■ BHRAMANANTDARSHANA

A Quantum Computing Education Platform

Complete Project Documentation & Presentation Guide

Project Type:	Quantum Computing Simulator + Interactive Frontend
Technology Stack:	C++17 Next.js 16 React 19.2 TypeScript Tailwind CSS
Status:	Active Development
Qubits Supported:	1-5 (Frontend), n-qubit (Backend)
Gates Implemented:	15+
Pre-built Templates:	10+
Date:	November 07, 2025

For presentation preparation by: Bhuvnesh

Generated: November 07, 2025 at 20:59

■ Table of Contents

- 1. Executive Overview
- 2. Project Architecture
- 3. Core Components
- 4. Technology Stack
- 5. Detailed Features
- 6. User Guide
- 7. Technical Implementation
- 8. Installation & Deployment
- 9. Future Roadmap
- 10. Learning Outcomes

1. Executive Overview

What is Bhramanantdarshana?

Bhramanantdarshana (Sanskrit: 'witnessing the infinite circulation of cosmic energy') is a comprehensive quantum computing education platform consisting of: (1) C++ Quantum Simulator Backend - a lightweight, didactic quantum register simulator built from first principles; (2) Next.js Interactive Frontend - a modern web interface for visual circuit building and simulation; (3) Educational Documentation - comprehensive guides and learning materials.

Project Goals

- Demystify Quantum Computing through transparent, step-by-step implementation
- Provide Visual Learning with interactive, drag-and-drop circuit building
- Focus on Pedagogy over performance
- Demonstrate Algorithmic Quantum Advantage with working examples
- Eliminate External Dependencies (backend uses pure C++17, no quantum SDKs)

Key Statistics

Qubits Supported	1-5 (frontend), n-qubit (backend)
Time Steps per Circuit	8
Quantum Gates	15+ (H, X, Y, Z, S, T, CNOT, SWAP, M)
Pre-built Templates	10+ educational circuits
Documentation	5+ comprehensive guides
Code Philosophy	Pedagogical with extensive comments

2. Project Architecture

System Design Overview

Bhramanantdarshana consists of two main components that work together: a C++ backend providing the quantum simulation engine, and a Next.js frontend offering visual circuit building and real-time visualization. The architecture prioritizes clarity and educational value over raw performance.

Backend Architecture (C++)

Quantum_registers_cl.cpp: Core state vector management, normalization, measurement

RegisterGates_cl.cpp: Gate operations on quantum registers

Gates_cl.cpp: Single-qubit gate definitions and theory

DeutscheAlgo_example.cpp: Deutsch algorithm implementation with oracles

Frontend Architecture (Next.js/React)

QuantumCircuitBuilderV2.tsx: Main application logic and state management

CircuitGridWithConnections.tsx: Drop zones and visual connection lines

GateLibrary.tsx: Draggable gate palette with descriptions

QubitVisualizer.tsx: Bloch sphere and probability visualization

TemplateGallery.tsx: Pre-built educational circuits

Toast.tsx: User notifications

3. Core Components

Backend Components: The Quantum Engine

State Vector Representation

The quantum state is stored as a vector of 2^n complex amplitudes, where n is the number of qubits. Each amplitude corresponds to a basis state, with the amplitude's squared magnitude representing the measurement probability for that state.

Gate Operations

Single-qubit gates work by pairing amplitudes based on bit positions and applying the gate matrix. Multi-qubit gates like CNOT create entanglement by conditionally modifying state elements.

Measurement Mechanics

Measurement converts quantum probabilities into classical outcomes through probabilistic collapse. The simulator supports both destructive measurement (which collapses the state) and non-destructive sampling (for statistics gathering).

Frontend Components: The User Interface

Circuit Builder

The main interface where users select qubits, drag gates from a library, and place them on a circuit. Supports 1-5 qubits with 8 time steps each. Real-time visualization of quantum states as gates are added.

Two-Qubit Connections

Interactive workflow for creating entangling gates. User drags a CNOT/SWAP, drops it on control qubit, then clicks target qubit to complete the connection. Visual lines show the relationship.

Real-Time Visualization

Multiple representation formats: Bloch sphere (geometric), probability bars (numerical), state vector (mathematical), and measurement results (classical outcomes). Helps learners understand from multiple perspectives.

4. Technology Stack

Component	Technology	Version	Purpose
Backend	C++	C++17	Core quantum simulation
Backend	STL	Standard	Containers & algorithms
Backend	Compiler	MinGW/g++	Compilation
Frontend	Next.js	16.x	React framework
Frontend	React	19.2.x	UI library
Frontend	TypeScript	5.x	Type safety
Frontend	Tailwind CSS	4.x	Styling
Frontend	react-dnd	Latest	Drag & drop
Viz	Python	3.8+	Data visualization
Viz	matplotlib	Latest	Plotting library
VCS	Git	Latest	Version control

5. Detailed Features

Feature 1: Interactive Drag-and-Drop Building

User Journey: Select qubit count → Drag gates from library → Drop on circuit grid → Build complete quantum circuit → Run simulation → View results.

Implementation: Uses React DnD with HTML5 backend for smooth drag-and-drop. Drop zones update state immediately. Visual feedback shows gate placement in real-time.

Feature 2: Two-Qubit Gate Connections

Workflow: Drag CNOT/SWAP → Drop on control qubit → System prompts for target qubit → Click target qubit → Connection line drawn → Circuit ready.

Teaching Value: Step-by-step interaction helps learners understand multi-qubit gate mechanics. Visual connection lines demonstrate entanglement relationships.

Feature 3: Pre-built Circuit Templates (10+)

■ **Entanglement:** Bell State, GHZ State, Entanglement + Measurement

■ **Algorithms:** Deutsch Algorithm (quantum advantage demonstration)

■ **Gate Demos:** Quantum NOT, Superposition, Phase Flip, Pauli Gates, State Swap

■ **Measurement:** Measurement Collapse, Entangled Measurement

Each template includes pre-configured gates, connections, and detailed explanation. Users can load instantly and modify to experiment.

Feature 4: Real-Time Visualization

Bloch Sphere: Geometric representation of single-qubit states

Probability Bars: Visual $|0\rangle$ and $|1\rangle$ probability distribution

State Vector: Mathematical notation of quantum superposition

Entanglement Badge: Shows which qubits are entangled

Measurement Results: Classical outcomes from simulation

6. User Guide

Quick Start: 5 Minutes to First Circuit

1. Navigate to <http://localhost:3000>
2. Application loads with interactive circuit builder
3. Keep default 1 qubit, or select 2-5 from dropdown
4. Drag 'H' (Hadamard) gate to qubit 0, time step 0
5. Click 'Run Simulation' button
6. Watch Bloch sphere update showing superposition
7. See 50% probability for $|0\rangle$ and $|1\rangle$

Common Tasks

Task: Create Superposition

1. Keep 1 qubit.
2. Drag 'H' gate to position [0,0].
3. Run simulation.
4. Result: 50% chance of measuring 0 or 1.

Task: Create Bell State (Entanglement)

1. Select 2 qubits.
2. Drag 'H' to [0,0].
3. Drag 'CNOT' to [0,1].
4. Click [1,1] to complete connection.
5. Run. Result: Qubits always measure same value.

Task: Load Pre-built Template

1. Click '■ Templates' button (bottom-right).
2. Browse available circuits.
3. Click 'Load →' on desired template.
4. Circuit auto-loads with explanation.

7. Technical Implementation

Quantum State Representation

State Vector Encoding: For n qubits, we maintain 2^n complex amplitudes. Index i represents the binary state $|b_n \dots b_2 b_1 b_0\rangle$. For example, in a 3-qubit system: index 0 = $|000\rangle$, index 5 = $|101\rangle$.

Normalization: After each gate operation, the sum of squared magnitude of all amplitudes must equal exactly 1.0. This represents the conservation of probability.

Gate Implementation Pattern

Gates work by pairing amplitudes based on the target qubit's bit position. For each amplitude at index i with a 0 at the target position, we find the paired index j (with 1 at target position). We then apply the gate matrix to both amplitudes simultaneously.

Example (Hadamard): For each pair of amplitudes, apply the matrix $\frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$. This creates superposition by mixing the two basis state amplitudes.

Measurement Mechanics

Collapse Process: 1) Calculate probability for each outcome: $P(i) = |\text{amplitude}[i]|^2$. 2) Create cumulative distribution. 3) Sample using uniform random number. 4) Zero all other amplitudes. 5) Renormalize to probability 1.

Non-Destructive Sampling: Same as above but without modifying state—used for statistics gathering.

Two-Qubit Gate Mechanics

CNOT Gate: Controlled-NOT flips the target qubit if control is $|1\rangle$. Implemented by iterating through state amplitudes and swapping conditionally based on control qubit value.

SWAP Gate: Exchanges states between two qubits. Implemented by permuting amplitude positions.

8. Installation & Deployment

Backend Setup (C++)

Compilation (Windows with MinGW):

```
g++ -std=c++17 Quantum_registers_cl.cpp RegisterGates_cl.cpp DeutscheAlgo_example.cpp -o deutsch.exe
```

Execution:

```
./deutsch.exe
```

Frontend Setup (Next.js)

1. `cd quantum-frontend`
2. `npm install`
3. `npm run dev`
4. Visit <http://localhost:3000>

Production Build

`npm run build` creates optimized production build. `npm run start` launches production server.

9. Future Roadmap

Phase 1: Near Term (0-3 months)

- Backend: Better random number generation (std::mt19937)
- Backend: Unit tests for all gate operations
- Frontend: Additional gate types (T, S, Rx, Ry, Rz)
- Frontend: 3D Bloch sphere visualization
- Frontend: Circuit save/load as JSON
- Documentation: Video tutorials

Phase 2: Medium Term (3-6 months)

- Algorithm: Deutsch-Jozsa (full implementation)
- Algorithm: Grover's search algorithm
- Algorithm: Bernstein-Vazirani algorithm
- Feature: Quantum state tomography
- Feature: Entanglement entropy calculation

Phase 3: Long Term (6-12 months)

- Platform: Cloud hosting (Vercel, AWS)
- Platform: User accounts and circuit sharing
- Physics: Quantum error correction basics
- Physics: Noise simulation models
- Gamification: Achievement badges and leaderboards

10. Learning Outcomes

For Students Using This Platform:

- ✓ Quantum state vector representation and complex amplitudes
- ✓ Superposition principle and measurement collapse
- ✓ Single-qubit gates (H, X, Y, Z, S, T) and their effects
- ✓ Multi-qubit entanglement (CNOT, SWAP gates)
- ✓ Quantum algorithms demonstrating computational advantage
- ✓ Bell states and GHZ states for entanglement study
- ✓ Measurement correlation in entangled systems
- ✓ Linear algebra applications in quantum computing

For Educators Using This Platform:

- ◆ Visual demonstration tool for abstract quantum concepts
- ◆ Interactive experimentation environment for students
- ◆ Ready-made circuit templates for structured lessons
- ◆ Immediate feedback and visual confirmation of concepts
- ◆ Customizable templates for problem-based learning

Key Competencies Developed:

Competency	Level	How Demonstrated
Quantum Mechanics	Foundational	Building and simulating circuits
Algorithm Design	Intermediate	Deutsch algorithm implementation
Linear Algebra	Intermediate	Gate matrices and operations
Programming	Intermediate	Interactive circuit building
Problem Solving	Advanced	Designing quantum circuits

Conclusion

Bhramanantdarshana represents a comprehensive effort to democratize quantum computing education. By combining a transparent C++ backend implementation with an intuitive interactive frontend, the platform enables learners to understand quantum mechanics from first principles.

The project serves educators, students, and enthusiasts by providing:

- Clear Implementation: Every quantum concept from first principles
- Visual Learning: Interactive UI supporting multiple learning styles
- Documented Examples: 10+ pre-built circuits demonstrating concepts
- Real Algorithms: Deutsch algorithm showing quantum advantage
- Extensible: Designed for addition of new gates and algorithms

This documentation provides everything needed for Bhuvnesh to create a compelling presentation of the Bhramanantdarshana quantum computing education platform.

Generated: November 07, 2025 at 20:59

Project Status: Active Development